

矩阵的舞蹈

多维数组与矩阵模式 · 12题 · C++ & Python 双语对照

第四周结束，小鲁已经能熟练地操作一维数组了。但小华告诉他："一维数组只是第一步——真实世界的的数据很少是一条线。想想电子表格、数码照片、游戏棋盘.....这些都是二维的。"小鲁若有所思："所以.....我需要学会在行列之间跳舞？"

本章12道题分三个板块：**区域遍历**（NQ055-063）——学会用行列关系精确切割矩阵区域；**模式填充**（NQ059,064,065）——用数学公式一行替代几十行循环，体验"数学建模"的力量；**蛇形遍历**（NQ066）——方向数组的经典实战，贪吃蛇游戏引擎的底层算法。

本章角色

-  **小鲁**（初学者）· 正在经历从"逐题应付"到"寻找规律"的思维跃迁
-  **小华**（算法高手）· ACM竞赛队队长，用"对称性"一句话解决四道相似的题
-  **小栋**（工程师）· 把枯燥的矩阵遍历与游戏引擎、图像处理等实际应用联系起来
-  **小嘉**（校主精神）· 带着"同心方"从南洋归来，用数学和编程的交汇点启发学生

矩阵三角区域速查表（12×12矩阵）

- ↗ **右上三角**： $j > i$ （不含对角线，66个元素）
- ↖ **左上三角**： $j < 11-i$ （不含反对角线，66个元素）
- ↙ **左下三角**： $j < i$ （不含对角线，66个元素）
- ↘ **右下三角**： $i+j > 10$ （66个元素）
- ↑ **上方区域**： $i < j$ 且 $i+j < 11$ （30个元素）
- ↓ **下方区域**： $i > j$ 且 $i+j > 10$ （30个元素）
- ← **左方区域**： $j < i$ 且 $i+j < 11$ （30个元素）
- **右方区域**： $j > i$ 且 $i+j > 10$ （30个元素）

核心心法：四个三角区域通过两种对角线分割（主对角线、反对角线），四个"口"字区域是两个三角的交集。掌握对称性=一道题的功夫解决四道题。

NQ055 数组的右上半部分 AcWing 745

小鲁拿着一支荧光笔，在一张12×12的格子纸上涂色：右上三角。" $j>i$ ——第0行从第1列到第11列，第1行从第2列到第11列....."小鲁一边涂一边念叨。小华走过来："你在画什么？""我在数右上三角有多少个元素！总共12×12减去对角线12，再除2——就是66个。"小华笑道："你已经发现了三角区域的核心：找到行和列的关系表达式，剩下的循环就水到渠成。"

输入 第一行操作类型(S/M)。然后144个浮点数（逐行读入）。

输出 结果保留1位小数。S输出和，M输出平均值。

范围 矩阵元素 -10^6 到 10^6

输入

S
(144个浮点数...)

输出

(sum,1 decimal)

思路 右上三角： $j > i$ 。双层循环遍历矩阵，当 $j > i$ 时累加。不包括对角线。66个元素。S求总和，M求平均时除以66（注意不是144）。区域遍历是矩阵题的灵魂——关键在于找到正确的行列关系表达式。

技巧 右上三角条件： $j > i$ 。共有元素数 $= (12 \times 12 - 12) / 2 = 66$ 。C++用double型累加，注意不要用int。Python的float是双精度。

C++

```
#include <cstdio>

int main()
{
    char t;
    scanf("%c", &t);
    double a[12][12];

    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            scanf("%lf", &a[i][j]);

    int c = 0;
    double s = 0;

    for (int i = 0; i < 12; i++)
        for (int j = i + 1; j < 12; j++)
        {
            c++;
            s += a[i][j];
        }

    if (t == 'S') printf("%.1lf\n", s);
    else printf("%.1lf\n", s / c);

    return 0;
}
```

Python

```
l=input();t=input().strip();s=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if j>i:s+=x
print(f'{s if t=="S" else s/66:.1f}')
```

NQ056 数组的左上半部分 AcWing 747

小鲁转向左上三角：“这次应该用 $j < 11 - i$ 吧？反对角线以上……”小华在纸上画了一笔：“看，反对角线满足 $i + j = 11$ 。左上三角就是 $j < 11 - i$ 。但还有另外一种理解——它和右上三角本质上是同一个形状，只是参考线不一样。一个用主对角线($j > i$)，一个用反对角线($j < 11 - i$)。两题放一起对比，四种三角区域的条件就清晰了。”

输入

操作类型(S/M)+144浮点数。

输出

保留1位小数。

范围

同NQ055

输入

输出

S (sum,1 decimal)
(144...)

思路 左上三角： $j < 11 - i$ （反对角线以上）。反对角线条件： $i + j = 11$ 。元素数量也是66。将四个三角区域的遍历条件对比理解，是掌握矩阵区域操作的关键—— $j > i$ （右上）、 $j = 11$ （右下）。

技巧 反对角线条件： $i + j = n - 1$ （ $n = 12$ 时为11）。左上三角用 $<$ 。四个区域条件对称且互补，理解这一规律后所有矩阵三角题迎刃而解。

C++

```
#include <stdio>

int main()
{
    double a[12][12];
    char t;

    scanf("%c", &t);
    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            scanf("%lf", &a[i][j]);

    int c = 0;
    double s = 0;
    for (int i = 0; i < 12; i++)
        for (int j = 0; j <= 10 - i; j++)
        {
            c++;
            s += a[i][j];
        }

    if (t == 'S') printf("%.1lf\n", s);
    else printf("%.1lf\n", s / c);

    return 0;
}
```

Python

```
t=input().strip();s=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if j<11-i:s+=x
    print(f'{s if t=="S" else s/66:.1f}')
```

NQ057 数组的上方区域 AcWing 749

"等等——'上方区域'又是什么？"小鲁挠着头。小栋在屏幕上画了一个"八"字形："你看，把 12×12 矩阵的上半部分，去掉左边和右边的两个三角——剩下的就是上方区域。它其实是两个条件的交集：既在主对角线之上(i

输入 操作类型(S/M)+144浮点数。

输出 保留1位小数。

范围 同NQ055

输入

S
(144...)

输出

(sum,1 decimal)

思路 上方区域是两个三角的交集：i

技巧 交集条件：i

C++

```
#include <stdio>
#include <iostream>

using namespace std;

int main()
{
    char t;
    cin >> t;
    double m[12][12];

    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 0; i < 5; i++)
        for (int j = i + 1; j <= 10 - i; j++)
        {
            c += 1;
            s += m[i][j];
        }
    if (t == 'M') printf("%.1lf\n", s / c);
    else printf("%.1lf\n", s);

    return 0;
}
```

Python

```
t=input().strip();s=c=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if i<j and i+j<11:s+=x;c+=1
    print(f'{s if t=="S" else s/c:.1f}')
```

NQ058 数组的左方区域 AcWing 751

"左边区域呢？"小鲁问。小华说："对称的！把上方区域的j>i改成j

输入 操作类型(S/M)+144浮点数。

输出 保留1位小数。

范围 同NQ055

输入
S
(144...)

输出
(sum,1 decimal)

思路 左方区域与上方区域对称：将条件改为j

技巧 左右对称：互换i和j的角色。上方=去掉左右三角的上部，左方=去掉上下三角的左部。对称思维是编程中的强大工具——写一遍代码，改一行条件就能得到四种区域的遍历。

C++

```
#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    char t;
    cin >> t;

    double m[12][12];
    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 1; i <= 5; i++)
        for (int j = 0; j <= i - 1; j++)
        {
            s += m[i][j];
            c += 1;
        }
    for (int i = 6; i <= 10; i++)
        for (int j = 0; j <= 10 - i; j++)
        {
            s += m[i][j];
            c += 1;
        }
    if (t == 'M') printf("%.1lf\n", s / c);
    else printf("%.1lf\n", s);

    return 0;
}
```

Python

```
t=input().strip();s=c=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if j<i and i+j<11:s+=x;c+=1
    print(f'{s if t=="S" else s/c:.1f}')
```

NQ059 平方矩阵I AcWing 753

小嘉走进教室，手里拿着一张回字形的图纸：“这在南洋叫‘同心方’——外面一圈是1，往里一圈是2，再往里是3……最中心的值是 $\min(N+1, N+1, N, N)+1$ 。”小鲁看得目瞪口呆：“这么复杂的图案，用一个公式就能生成？！”小华解释：“每个位置的值=到四条边的最近距离+1。外层距离0→值1，往里一层距离1→值2。理解这个‘距离模型’比写100行if更重要。”

输入

多个整数N，以0结束。

输出

每个矩阵占N行，每个数占3字符宽度右对齐。矩阵间空行。

范围

 $1 \leq N \leq 100$

输入

1
2
3
0

输出

1

1 1
1 1

1 1 1

```
1 2 1
1 1 1
```

思路 数学公式法： $val = \min(i, j, n-1-i, n-1-j) + 1$ (0-indexed)。这个公式的物理意义：每个位置的值等于它到四条边的最短距离加1。最外层距离为0→值=1，往里一层距离为1→值=2，以此类推。理解公式比手写模拟更体现编程智慧。

技巧 到四条边的距离：上i、左j、下n-1-i、右n-1-j。取最小值+1即为该位置的值。这个公式体现了"数学建模"的力量——用数学语言描述复杂的几何规律。

C++

```
#include <iostream>

using namespace std;

int main()
{
    int n;
    while (cin >> n, n)
    {
        for (int i = 1; i <= n; i++)
        {
            for (int j = 1; j <= n; j++)
            {
                int up = i, down = n - i + 1, left = j, right = n - j + 1;
                cout << min(min(up, down), min(left, right)) << ' ';
            }
            cout << endl;
        }

        cout << endl;
    }

    return 0;
}
```

Python

```
while True:
    n = int(input())
    if n == 0: break
    for i in range(n):
        row = []
        for j in range(n):
            v = min(i, j, n-1-i, n-1-j) + 1
            row.append(f'{v:3d}')
        print(' '.join(row))
    print()
```

NQ060 数组的右下半部分 AcWing 748

小鲁拿着一面镜子放在矩阵的左上角——"如果从左上三角对称过去……"小华笑道："没错！右上三角($j > i$)的对称是左下三角($j < i$)。四题八遍的遍历模式，如果用'条件表达式'分类，一共就两种本质：主对角线分割和反对角线分割。"

输入 操作类型(S/M)+144浮点数。

输出 保留1位小数。

范围 同NQ055

输入

输出

(sum, 1 decimal)

S
(144...)

思路 右下三角： $i+j>10$ 。与NQ056的左上三角($i+j<11$)对称。通过前6个矩阵三角题的系统训练，你应该已经掌握了区域遍历的核心技巧：找到正确的行列关系表达式。

技巧 $i+j>10$ 与 $11-i-1$ 通。矩阵遍历的4个三角区域至此全部覆盖。建议画一个5×5矩阵逐格验证——对着纸面比对着屏幕理解更快。

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    char t;
    cin >> t;
    double m[12][12];

    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 1; i <= 11; i++)
        for (int j = 12 - i; j <= 11; j++)
        {
            s += m[i][j];
            c += 1;
        }

    if (t == 'S') printf("%.1lf\n", s);
    else printf("%.1lf\n", s / c);

    return 0;
}
```

Python

```
t=input().strip();s=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if i+j>10:s+=x
    print(f'{s if t=="S" else s/66:.1f}')
```

NQ061 数组的左下半部分 AcWing 746

"左下三角就是ji对称，只是方向反过来。"小鲁总结说。小华欣慰地拍拍他："你已经从'逐题应付'升级到'找规律'了。记住这种感觉——以后学任何新算法，第一步都是找pattern：有没有对称性？能不能归约成已知问题？这是从学生到工程师的思维跃迁。"

输入 操作类型(S/M)+144浮点数。

输出 保留1位小数。

范围 同NQ055

输入

输出

S (sum,1 decimal)
(144...)

思路 左下三角：j)i)对称。遍历条件简单明了——i和j的大小关系决定位置归属。注意不包括主对角线。

技巧 四个三角区域的条件总结：右上j>i、左下j10-i。掌握这4个条件，所有标准矩阵三角题都不在话下。

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    char t;
    cin >> t;

    double m[12][12];
    for (int i = 0; i < 12; i ++ )
        for (int j = 0; j < 12; j ++ )
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 0; i < 12; i ++ )
        for (int j = 0; j <= i - 1; j ++ )
        {
            s += m[i][j];
            c += 1;
        }

    if (t == 'S') printf("%.1lf\n", s);
    else printf("%.1lf\n", s / c);

    return 0;
}
```

Python

```
t=input().strip();s=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if j<i:s+=x
    print(f'{s if t=="S" else s/66:.1f}')
```

NQ062 数组的下方区域 AcWing 750

"哎，下方区域的条件应该就是要i>j且j>10-i吧？"小鲁试着推导。"等一下，j>10-i就是i+j>10。"小栋纠正道。"对对对，i>j且i+j>10——和上方区域的i

输入 操作类型(S/M)+144浮点数。

输出 保留1位小数。

范围 同NQ055

输入
S
(144...)

输出
(sum,1 decimal)

思路 下方区域: $i > j$ 且 $i + j > 10$ 。与上方区域($i < j$ 且 $i + j > 10$)。

技巧 对称总结: 上方($i < j$ 且 $i + j > 10$)、左方($j < i$ 且 $i + j > 10$)。四个区域构成一个完整的"口"字形框架。

C++

```
#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    char t;
    cin >> t;

    double m[12][12];
    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 7; i <= 11; i++)
        for (int j = 12 - i; j <= i - 1; j++)
        {
            c += 1;
            s += m[i][j];
        }
    if (t == 'M') printf("%.1lf\n", s / c);
    else printf("%.1lf\n", s);

    return 0;
}
```

Python

```
t=input().strip();s=c=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if i>j and i+j>10:s+=x;c+=1
    print(f'{s if t=="S" else s/c:.1f}')
```

NQ063 数组的右方区域 AcWing 752

小鲁把四道区域题全部AC后,看着屏幕上的"Accepted",长出了一口气。"8道矩阵区域遍历题全部完成了!"小华说:"总结一下——你真正学到了什么?不是8个条件表达式,而是'精确描述几何区域的思维方式'。未来做图像处理、游戏碰撞检测、地图分析,用的还是这个思维——把视觉区域翻译成代码条件。"

输入 操作类型(S/M)+144浮点数。

输出 保留1位小数。

范围 同NQ055

输入

S
(144...)

输出

(sum,1 decimal)

思路 右方区域: $j > i$ 且 $i + j > 10$ 。30个元素。至此矩阵区域遍历全部完成。8道题的变化本质是条件表达式的组合。掌握了规律,任何形状的区域都能精确描述。

技巧 四个"口"字区域全部完成后，你的二维数组遍历能力已经达到面试水平。Key takeaway：精确的行列条件表达式=矩阵操作的全部。

C++

```
#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    char t;
    cin >> t;
    double m[12][12];

    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 1; i <= 5; i++)
        for (int j = 12 - i; j <= 11; j++)
        {
            s += m[i][j];
            c += 1;
        }
    for (int i = 6; i <= 10; i++)
        for (int j = i + 1; j <= 11; j++)
        {
            s += m[i][j];
            c += 1;
        }
    if (t == 'M') printf("%.1lf\n", s / c);
    else printf("%.1lf\n", s);

    return 0;
}
```

Python

```
t=input().strip();s=c=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if j>i and i+j>10:s+=x;c+=1
    print(f'{s if t=="S" else s/c:.1f}')
```

NQ064 平方矩阵II AcWing 754

"这道比上一道更简单！"小鲁看着平方矩阵II的模式：对角线是1，往外一层是2，再往外是3....."这不就是看每个位置到对角线的距离吗？"小华回答："对！ $\text{abs}(i-j)$ 就是到对角线的'曼哈顿距离'——对角线上 $\text{abs}(i-j)=0 \rightarrow$ 值 $0+1=1$ ，偏离一格 $\text{abs}=1 \rightarrow$ 值 $1+1=2$ 。两种平方矩阵展示了两种数学建模：一个是到四条边的距离，一个是到对角线的距离。"

输入 多个整数N（0结束）。

输出 $N \times N$ 矩阵，每个数占3字符宽。矩阵间空行。

范围 $1 \leq N \leq 100$

输入

3
0

输出

1 2 3
2 1 2

思路 $|i-j|+1$ 公式产生以主对角线为对称轴的矩阵。对角线上 $|i-j|=0 \rightarrow$ 值=1，离对角线每远一步值增加1。这比平方矩阵I的“四边距离”公式更简单，展示了另一种数学建模方式。

技巧 $\text{abs}(i-j)+1$ 。C++用stdlib的abs()，Python内置abs()。注意绝对值函数对负数的处理——自动取正。这种“距离模型”在矩阵题中非常常见。

C++

```
#include <iostream>

using namespace std;

int q[100][100];

int main()
{
    int n;
    while (cin >> n, n)
    {
        for (int i = 0; i < n; i++)
        {
            q[i][i] = 1;
            for (int j = i + 1, k = 2; j < n; j++, k++) q[i][j] = k;
            for (int j = i + 1, k = 2; j < n; j++, k++) q[j][i] = k;
        }

        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++) cout << q[i][j] << ' ';
            cout << endl;
        }
        cout << endl;
    }

    return 0;
}
```

Python

```
while True:
    n=int(input())
    if n==0:break
    for i in range(n):
        print(''.join(f'{abs(i-j)+1:3d}' for
j in range(n)))
    print()
```

NQ065 平方矩阵III AcWing 755

小鲁看着第三个平方矩阵： $2^0, 2^1, 2^2 \dots$ “这是指数增长！”“对！”小华说，“两个技巧：第一，用位运算 $1 < (i+j)$ 比 $\text{pow}(2, i+j)$ 快很多；第二， $N \leq 15$ 的限制是精心设计的—— $N=15$ 时最大值 $2^{28} \approx 2.68$ 亿，刚好在 int 32 范围内。如果 N 再大就要用 long long 甚至大整数了。题目设计者考虑得很周全。”

输入 多个整数 N （0 结束）。

输出 每个数右对齐到最大数的宽度+1。矩阵间空行。

范围 $1 \leq N \leq 15$

输入

3
0

输出

1 2 4
2 4 8
4 8 16

思路 $2^{(i+j)}$ 模式。C++用 $1<<(i+j)$ 位运算高效计算2的幂。Python可以 $2^{**}(i+j)$ 或 $1<<(i+j)$ 。右对齐宽度=最长数字的位数+1。注意 $N \leq 15$ 的限制： $N=15$ 时 $2^{28} \approx 2.68 \times 10^8$ (9位数)。

技巧 C++的 $1<$

C++

```
#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    int n;
    while (cin >> n, n)
    {
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                int v = 1;
                for (int k = 0; k < i + j; k++) v *= 2;
                cout << v << " ";
            }
            cout << endl;
        }

        cout << endl;
    }

    return 0;
}
```

Python

```
while True:
    n=int(input())
    if n==0:break
    w=len(str(2**(2*n-2)))+1
    for i in range(n):
        print(''.join(f'{2**(i+j):{w}d}' for
j in range(n)))
    print()
```

NQ066 蛇形矩阵 AcWing 756

"最后一道是大boss——蛇形矩阵！"小鲁聚精会神地盯着屏幕。小华拿出了他的'镇山法宝'："四向数组： $dx=[0,1,0,-1]$ ， $dy=[1,0,-1,0]$ ——分别代表右、下、左、上。从(0,0)出发向右走，撞墙或遇到已填的格子就顺时针转90度。方向数组是网格类问题的屠龙刀。"小栋补充道："其实你玩的贪吃蛇游戏引擎，底层就是这个算法。"

输入 一行两个整数n和m。

输出 n行m列的蛇形矩阵。

范围 $1 \leq n, m \leq 100$

输入

3 3

输出

```

1 2 3
8 9 4
7 6 5

```

思路 方向数组+边界检测。dx=[0,1,0,-1], dy=[1,0,-1,0]依次控制→↓←↑。撞墙或已填则转向d=(d+1)%4。while循环填充所有格子。方向数组是解决螺旋/蛇形/遍历类问题的通用模板——只需改变方向顺序就能实现不同的遍历模式。

技巧 方向数组是处理网格遍历的万能钥匙。记住4个方向的顺序和dx/dy的对应关系。Python中提前分配二维数组: [[0]*m for _ in range(n)] (注意不能用[[0]*m]*n——那是浅拷贝的经典陷阱!)。

C++

```

#include <iostream>

using namespace std;

int res[100][100];

int main()
{
    int n, m;
    cin >> n >> m;

    int dx[] = {0, 1, 0, -1}, dy[] = {1, 0, -1, 0};

    for (int x = 0, y = 0, d = 0, k = 1; k <= n * m; k++)
    {
        res[x][y] = k;
        int a = x + dx[d], b = y + dy[d];
        if (a < 0 || a >= n || b < 0 || b >= m || res[a][b])
        {
            d = (d + 1) % 4;
            a = x + dx[d], b = y + dy[d];
        }
        x = a; y = b;
    }

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++) cout << res[i][j] <<
        ' ';
        cout << endl;
    }

    return 0;
}

```

Python

```

n,m=map(int,input().split());a=[[0]*m
for _ in range(n)]
dx=[0,1,0,-1];dy=[1,0,-1,0];d=0;x=y=0
for v in range(1,n*m+1):
    a[x][y]=v;nx=x+dx[d];ny=y+dy[d]
    if nx<0 or nx>=n or ny<0 or ny>=m or
a[nx][ny]:d=(d+1)%4;nx=x+dx[d];ny=y+dy[d]
    x,y=nx,ny
for r in a:print(' '.join(map(str,
r)))

```

本章知识点总结

- 8种矩阵区域：4个三角（各66元素）+ 4个"口"字边（各30元素）= 精确遍历矩阵任意形状
- 3种填充模式：四边距离min公式、对角线距离abs公式、2的指数幂模式——用数学替代模拟
- 方向数组：dx=[0,1,0,-1], dy=[1,0,-1,0] → 控制四个方向旋转，蛇形/螺旋/遍历通用模板

- **对称思维**：上下、左右区域只是i和j角色的互换。找规律>逐题应付——从学生到工程师的思维跃迁
- **关键技巧**：Python二维列表[[0]*m for _ in range(n)]（不能用*操作符——那是浅拷贝陷阱！）
- **厦大精神**：自强不息，止于至善。12道矩阵题过后，你的多维数组能力已经达到面试水平。

— 第5章完 · 共12题 · 自强不息 —